

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 1

Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

3. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

4. Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link:

<https://archive.ics.uci.edu/dataset/267/banknote+authentication>

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

5. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Expected Output

Dataset summary and statistical information.

Task 2: Exploratory Data Analysis

Generate

- Histograms
- Correlation Heatmap
- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation

Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training

Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

Task 6: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

6. Source Code

Python Implementation

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from sklearn.model_selection import train_test_split
6 from sklearn.preprocessing import StandardScaler
7 from sklearn.metrics import accuracy_score

```

```

8 from sklearn.metrics import precision_score
9 from sklearn.metrics import recall_score
10 from sklearn.metrics import f1_score
11 from sklearn.metrics import confusion_matrix
12
13 !wget -O banknote_authentication.csv https://archive.ics.uci.edu/ml/machine-learning
14
15 import pandas as pd
16 column_names = [
17     "Variance",
18     "Skewness",
19     "Curtosis",
20     "Entropy",
21     "Class"
22 ]
23
24 df = pd.read_csv("banknote_authentication.csv", names=column_names)
25
26 from google.colab import files
27 files.download("banknote_authentication.csv")
28
29 df.head()
30 df.shape
31 df.info()
32 df.isnull().sum()
33 df.describe()
34
35 df.hist(figsize=(10, 8), bins=20)
36 plt.suptitle("Feature - Histograms")
37 plt.show()
38
39 plt.figure(figsize=(8,6))
40
41 sns.heatmap(
42     df.corr(),
43     annot=True,
44     cmap="coolwarm",
45     fmt=".2f"
46 )
47 plt.title("Correlation - Heatmap")
48 plt.show()
49
50 plt.figure(figsize=(8,6))
51 sns.scatterplot(
52     x="Variance",
53     y="Skewness",
54     hue="Class",
55     data=df

```

```

56 )
57 plt.title("Variance vs Skewness")
58 plt.show()
59
60 plt.figure(figsize=(12,6))
61 df.boxplot()
62 plt.title("Boxplots of Features")
63
64 X = df.drop("Class", axis=1)
65 y = df["Class"]
66 print(X.head())
67 print()
68 print(y.head())
69
70 from sklearn.model_selection import train_test_split
71 X_train, X_test, y_train, y_test = train_test_split(
72     X,y,
73     test_size=0.2,
74     random_state=42
75 )
76 print(X_train.shape)
77 print(X_test.shape)
78
79 from sklearn.preprocessing import StandardScaler
80 scaler = StandardScaler()
81 X_train = scaler.fit_transform(X_train)
82 X_test = scaler.transform(X_test)
83
84 print("Training Features:", X_train.shape)
85 print("Testing Features:", X_test.shape)
86
87 print("Training Labels:", y_train.shape)
88 print("Testing Labels:", y_test.shape)
89
90 class Perceptron:
91     def __init__(self, learning_rate=0.01, epochs=20):
92         self.learning_rate = learning_rate
93         self.epochs = epochs
94         self.weights = None
95         self.bias = None
96
97     def activation(self, x):
98         return 1 if x >= 0 else 0
99
100    def predict(self, x):
101        z = np.dot(x, self.weights) + self.bias
102        return self.activation(z)
103

```

```

104     def fit(self, X, y):
105         n_samples, n_features = X.shape
106         self.weights = np.zeros(n_features)
107         self.bias = 0
108         self.errors_history = []
109         self.weights_history = []
110         self.bias_history = []
111
112         for epoch in range(self.epochs):
113             errors = 0
114             for x_i, target in zip(X, y):
115                 prediction = self.predict(x_i)
116                 update = self.learning_rate * (target - prediction)
117                 self.weights += update * x_i
118                 self.bias += update
119
120                 if update != 0:
121                     errors += 1
122
123             self.errors_history.append(errors)
124             self.weights_history.append(self.weights.copy())
125             self.bias_history.append(self.bias)
126
127             print(f"Epoch-{epoch+1}/{self.epochs}")
128             print(f"Errors: -{errors}")
129             print(f"Weights: -{self.weights}")
130             print(f"Bias: - - -{self.bias}")
131             print("-" * 50)
132
133 model = Perceptron(
134     learning_rate=0.01,
135     epochs=20
136 )
137
138 model.fit(X_train, y_train)
139
140 y_pred = [model.predict(x) for x in X_test]
141
142 print("First 10 Predictions:", y_pred[:10])
143
144 from sklearn.metrics import (
145     accuracy_score,
146     precision_score,
147     recall_score,
148     f1_score,
149     confusion_matrix
150 )
151

```

```

152 accuracy = accuracy_score(y_test , y_pred)
153 precision = precision_score(y_test , y_pred)
154 recall = recall_score(y_test , y_pred)
155 f1 = f1_score(y_test , y_pred)
156 cm = confusion_matrix(y_test , y_pred)
157
158 print("Model-Evaluation")
159 print("_" * 30)
160 print(f"Accuracy:-{accuracy:.4f}")
161 print(f"Precision:-{precision:.4f}")
162 print(f"Recall:-{recall:.4f}")
163 print(f"F1-Score:-{f1:.4f}")
164
165 print("\nConfusion-Matrix")
166 print(cm)
167
168 plt.figure(figsize=(6,5))
169 sns.heatmap(
170     cm,
171     annot=True,
172     cmap="Blues",
173     fmt="d",
174     xticklabels=["Authentic", "Forged"],
175     yticklabels=["Authentic", "Forged"]
176 )
177 plt.xlabel("Predicted-Label")
178 plt.ylabel("Actual-Label")
179 plt.title("Confusion-Matrix")
180 plt.show()
181
182 plt.figure(figsize=(8,5))
183 plt.plot(
184     range(1, model.epochs + 1),
185     model.errors_history,
186     marker='o',
187     linewidth=2
188 )
189 plt.title("Training-Error-vs-Epoch")
190 plt.xlabel("Epoch")
191 plt.ylabel("Number-of-Misclassified-Samples")
192 plt.grid(True)
193 plt.show()
194
195 weights = np.array(model.weights_history)
196
197 plt.figure(figsize=(10,6))
198 for i in range(weights.shape[1]):
199     plt.plot(

```



```

200         range(1, model.epochs + 1),
201         weights[:, i],
202         marker='o',
203         label=f"Weight-{i+1}"
204     )
205
206 plt.title("Weight-Evolution")
207 plt.xlabel("Epoch")
208 plt.ylabel("Weight-Value")
209 plt.legend()
210 plt.grid(True)
211 plt.show()
212
213 plt.figure(figsize=(8,5))
214 plt.plot(
215     range(1, model.epochs + 1),
216     model.bias_history,
217     marker='o',
218     color='red',
219     linewidth=2
220 )
221 plt.title("Bias-Evolution")
222 plt.xlabel("Epoch")
223 plt.ylabel("Bias")
224 plt.grid(True)
225 plt.show()
226
227 learning_rates = [0.001, 0.01, 0.1]
228
229 plt.figure(figsize=(10,6))
230 for lr in learning_rates:
231     temp_model = Perceptron(
232         learning_rate=lr,
233         epochs=20
234     )
235     temp_model.fit(X_train, y_train)
236     plt.plot(
237         range(1, temp_model.epochs + 1),
238         temp_model.errors_history,
239         marker='o',
240         label=f"Learning-Rate-{lr}"
241     )
242
243 plt.title("Learning-Rate-Comparison")
244 plt.xlabel("Epoch")
245 plt.ylabel("Training-Errors")
246 plt.legend()
247 plt.grid(True)

```

7. Experimental Results

...	Variance	Skewness	Curtosis	Entropy	Class
count	1372.000000	1372.000000	1372.000000	1372.000000	1372.000000
mean	0.433735	1.922353	1.397627	-1.191657	0.444606
std	2.842763	5.869047	4.310030	2.101013	0.497103
min	-7.042100	-13.773100	-5.286100	-8.548200	0.000000
25%	-1.773000	-1.708200	-1.574975	-2.413450	0.000000
50%	0.496180	2.319650	0.616630	-0.586650	0.000000
75%	2.821475	6.814625	3.179250	0.394810	1.000000
max	6.824800	12.951600	17.927400	2.449500	1.000000

Figure 1: data description

```

Epoch 1/20
Errors : 47
Weights: [-0.06920384 -0.06634441 -0.06529563 -0.00524322]
Bias : -0.03
-----
Epoch 2/20
Errors : 22
Weights: [-0.07576842 -0.09439477 -0.05170347 -0.00312988]
Bias : -0.03
-----
Epoch 3/20
Errors : 28
Weights: [-0.08628609 -0.10389484 -0.06133036 -0.00617938]
Bias : -0.03
-----
Epoch 4/20
Errors : 24
Weights: [-0.09820347 -0.0939408 -0.08818487 -0.01037106]
Bias : -0.05
-----
Epoch 5/20
Errors : 29
Weights: [-0.10984895 -0.12095156 -0.08618021 0.0003308 ]
Bias : -0.04
-----
Epoch 6/20
Errors : 17
Weights: [-0.10385459 -0.13004831 -0.08702307 0.01372732]
Bias : -0.05
-----
Epoch 7/20
Errors : 26
Weights: [-0.10982533 -0.11869946 -0.11825568 -0.01997855]
Bias : -0.05
-----
Epoch 8/20
Errors : 19
Weights: [-0.12265141 -0.13640219 -0.09861007 -0.01184412]
Bias : -0.060000000000000005
-----
Epoch 9/20
Errors : 20
Weights: [-0.12083333 -0.14661863 -0.10078114 -0.00367508]
Bias : -0.060000000000000005
-----
Epoch 10/20
Errors : 22
Weights: [-0.12230111 -0.15079033 -0.1108229 0.00883899]
Bias : -0.060000000000000005
-----

```

Figure 2: evaluations

```

-----
Epoch 11/20
Errors : 20
...
Weights: [-0.14485453 -0.14504592 -0.11292681 -0.00241653]
Bias : -0.060000000000000005
-----
Epoch 12/20
Errors : 16
Weights: [-0.13904555 -0.15599948 -0.11550099 0.01452293]
Bias : -0.060000000000000005
-----
Epoch 13/20
Errors : 17
Weights: [-0.13971024 -0.14128072 -0.13272809 -0.0020794 ]
Bias : -0.07
-----
Epoch 14/20
Errors : 24
Weights: [-0.14383735 -0.14695168 -0.14245555 -0.00842043]
Bias : -0.07
-----
Epoch 15/20
Errors : 24
Weights: [-0.14069418 -0.17546993 -0.1259784 0.0013973 ]
Bias : -0.07
-----
Epoch 16/20
Errors : 15
Weights: [-0.13477927 -0.15902144 -0.14522405 -0.00813326]
Bias : -0.08
-----
Epoch 17/20
Errors : 23
Weights: [-0.13889593 -0.17379929 -0.14436365 0.00324382]
Bias : -0.07
-----
Epoch 18/20
Errors : 19
Weights: [-0.13993615 -0.16557246 -0.15027781 -0.01753637]
Bias : -0.08
-----
Epoch 19/20
Errors : 21
Weights: [-0.14269316 -0.18590916 -0.14056375 0.00228257]
Bias : -0.07
-----
Epoch 20/20
Errors : 18
Weights: [-0.15666011 -0.1846127 -0.14009741 -0.01316945]
Bias : -0.07
-----

```

Figure 3: evaluations

Model Evaluation

```

-----
Accuracy : 0.9818
Precision: 0.9841
Recall   : 0.9764
F1 Score : 0.9802

```

Confusion Matrix

```

[[146  2]
 [ 3 124]]

```

Figure 4: Model evaluations

8. Plots and interpretations

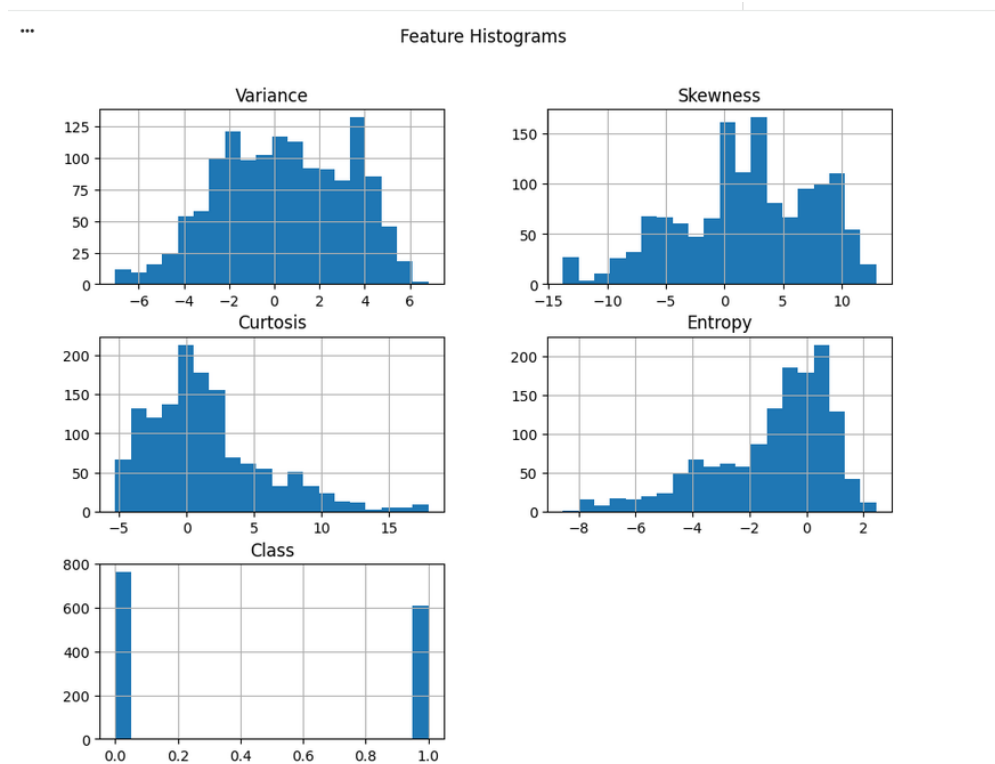


Figure 5: histograms

1. Feature Histograms The histograms show the distribution of each feature, indicating that the variables have different ranges and are not normally distributed. The class distribution is nearly balanced, making the dataset suitable for binary classification.

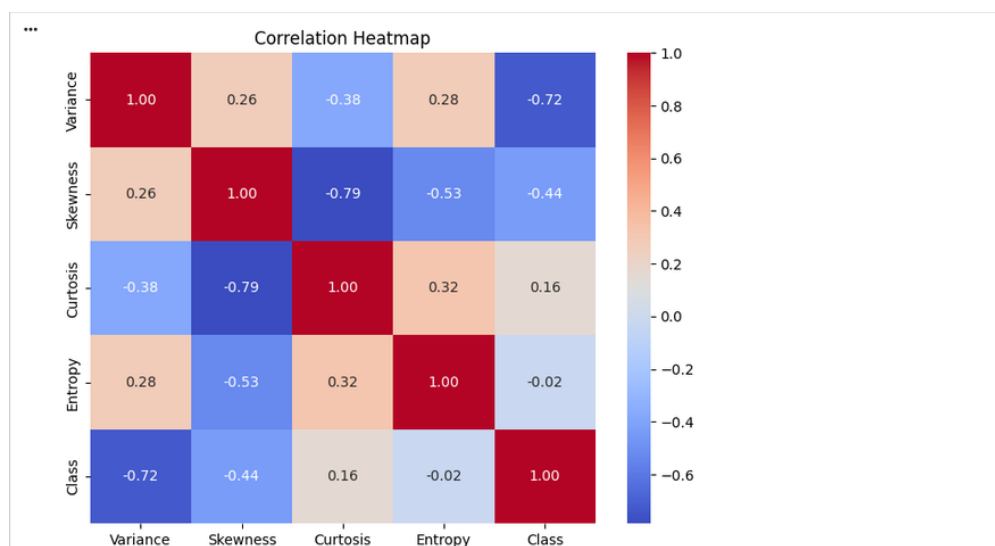


Figure 6: heatmap

2. Correlation Heatmap The heatmap shows the correlation between features, with Variance having the strongest negative correlation (-0.72) with the target class. This indicates that Variance is an important feature for distinguishing authentic and forged banknotes.

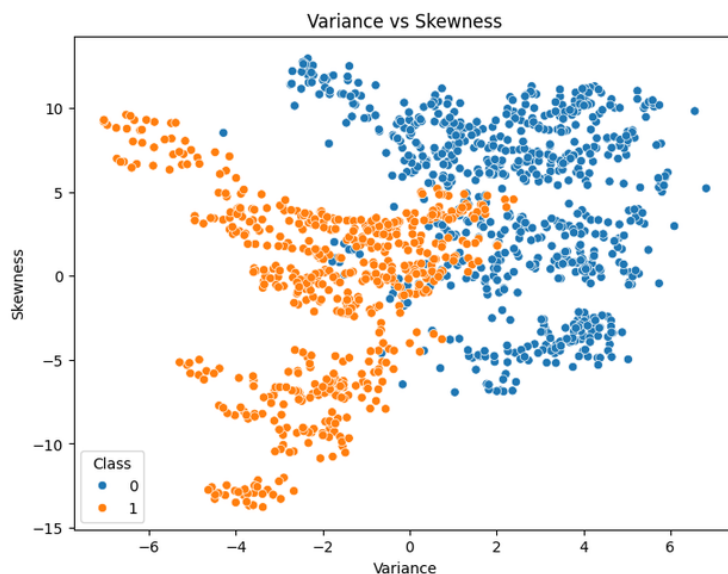


Figure 7: variance vs skewness

3. Scatter Plot (Variance vs Skewness) The scatter plot shows a noticeable separation between the two classes based on Variance and Skewness. Although there is slight overlap, these features provide good discriminative power for classification.

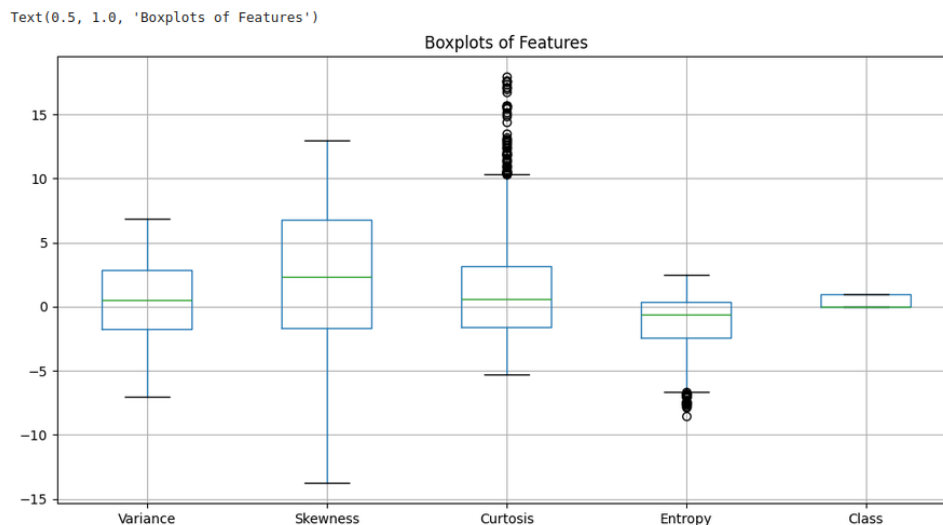


Figure 8: box plots

4. Boxplots of Features The boxplots reveal the spread and presence of outliers in the dataset, especially for the Curtosis feature. Despite the outliers, the feature distributions remain informative for classification.

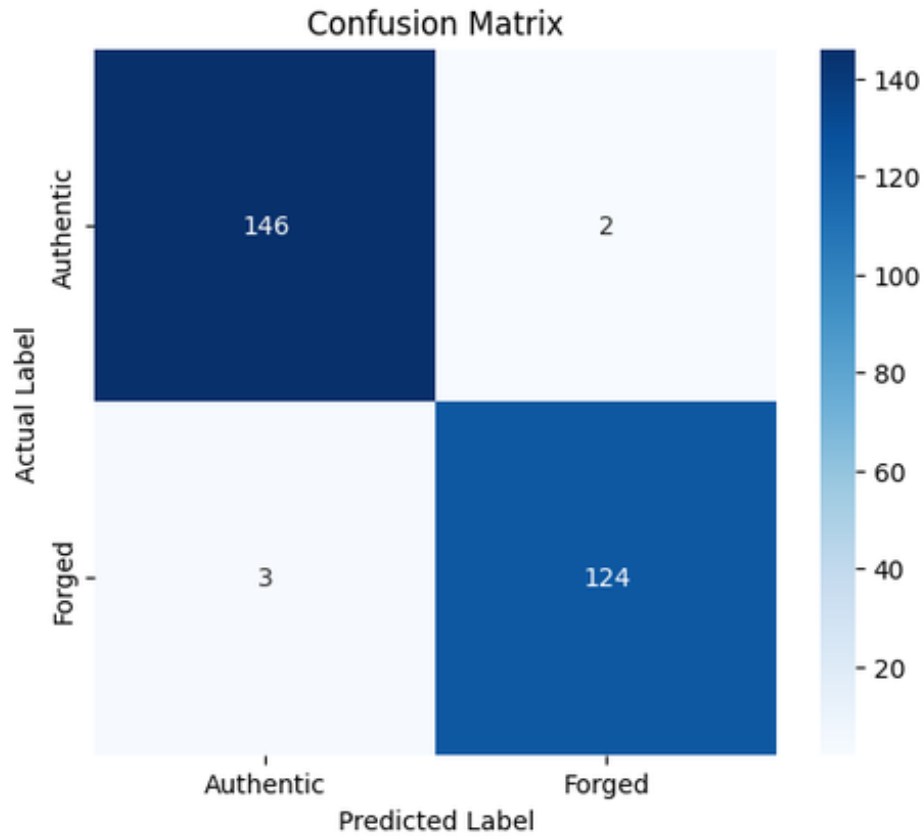


Figure 9: confusion matrix

5. Confusion Matrix The confusion matrix shows that the model correctly classified most samples, with only a few misclassifications. This demonstrates the high effectiveness of the perceptron in distinguishing authentic and forged banknotes.



Figure 10: training vs epoch

6. Training Error vs Epoch The training error decreases significantly during the initial epochs and then stabilizes, indicating that the perceptron gradually learns the decision boundary. The remaining fluctuations are due to difficult-to-classify samples.

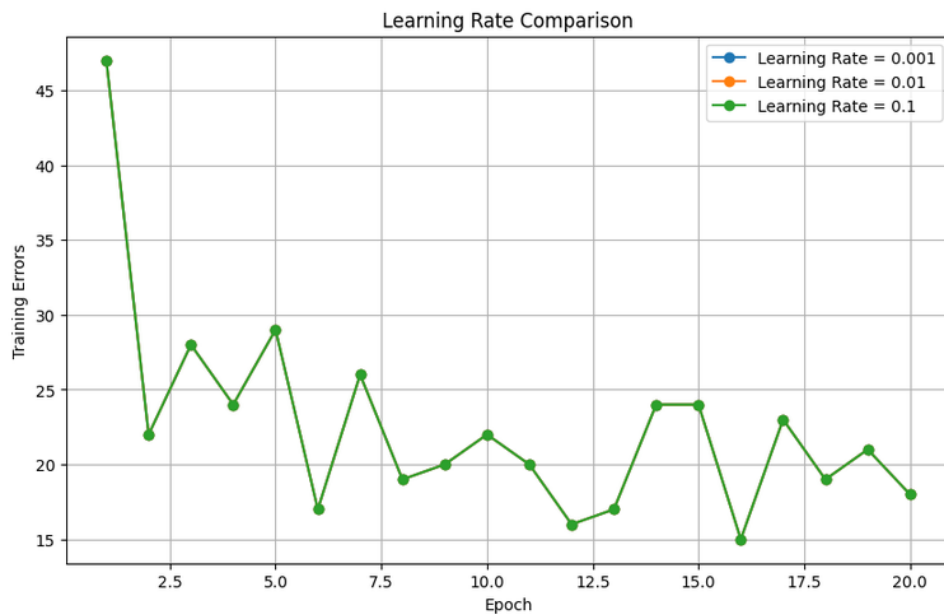


Figure 11: learningrate comparisons

7. Learning Rate Comparison The learning rate comparison shows that all learning rates converge to a similar training error, but at different speeds. A learning rate of 0.01 provides a good balance between convergence speed and training stability.

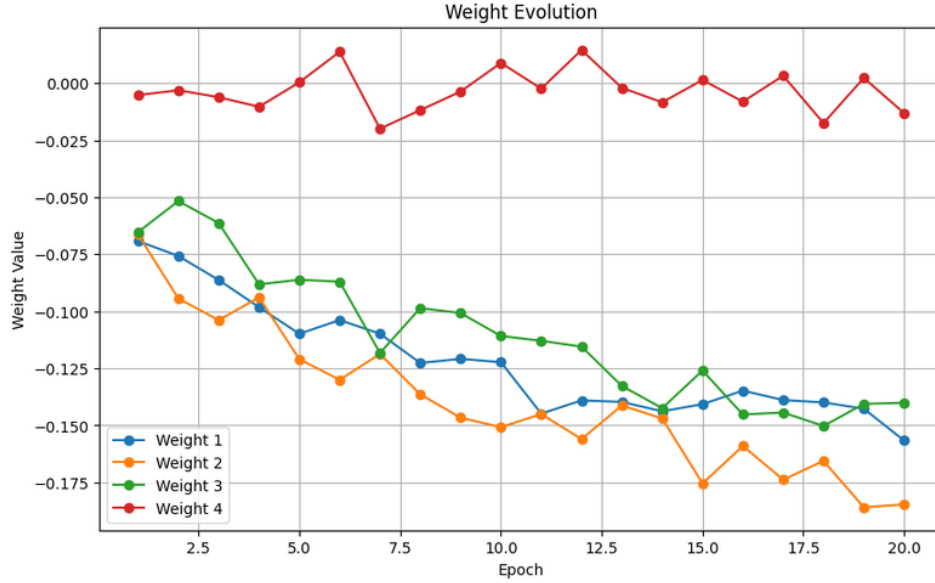


Figure 12: weight evolution

8. Weight Evolution The weights are updated continuously during training and gradually converge to stable values as the model learns. This indicates successful optimization of the perceptron's decision boundary.

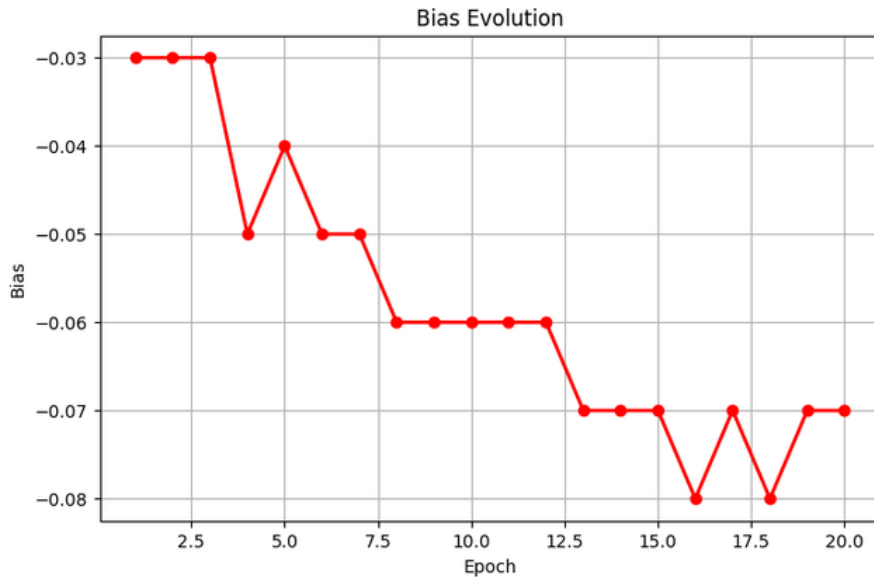


Figure 13: bias evolution

9. Bias Evolution The bias changes gradually across epochs and eventually stabilizes, indicating convergence of the learning process. The final bias value helps position the decision boundary for accurate classification.

9. Performance Tables

Table 1: Training Summary

Dataset Size	1372 samples
Train/Test Split	80% Training (1097), 20% Testing (275)
Learning Rate	0.01
Epochs	20
Final Weights	[-0.15641765 -0.1746633 -0.15518554 - 0.02596029]
Final Bias	-0.09
Accuracy	0.9636
Precision	1.0000
Recall	0.9213
F1-score	0.9590

Table 2: Epoch wise learning

Epoch	Errors	Weight 1	Weight 2	Weight 3	Weight 4	Bias
1/20	47	-0.072537	-0.064863	-0.064460	0.000199	-0.03
2/20	26	-0.082170	-0.093802	-0.051494	-0.005182	-0.03
3/20	34	-0.094676	-0.091400	-0.089055	-0.009565	-0.05
4/20	23	-0.110745	-0.095809	-0.106307	-0.013124	-0.04
5/20	27	-0.130964	-0.121796	-0.091128	-0.008771	-0.05
6/20	25	-0.139534	-0.128430	-0.104882	-0.006691	-0.04
7/20	19	-0.135270	-0.140369	-0.105269	-0.005157	-0.05
8/20	19	-0.133463	-0.144820	-0.104906	0.001333	-0.06
9/20	16	-0.134315	-0.131837	-0.123976	-0.011640	-0.06
10/20	26	-0.131687	-0.155810	-0.126006	0.011535	-0.06
11/20	25	-0.134304	-0.163344	-0.123422	0.004369	-0.07
12/20	26	-0.134814	-0.172105	-0.129577	0.001285	-0.07
13/20	21	-0.141143	-0.177991	-0.129746	0.005730	-0.08
14/20	21	-0.140347	-0.177649	-0.147387	0.006314	-0.07
15/20	22	-0.148292	-0.190448	-0.136416	0.000015	-0.07
16/20	16	-0.164249	-0.175786	-0.139463	-0.012613	-0.07
17/20	17	-0.149648	-0.171091	-0.157396	-0.018455	-0.08
18/20	15	-0.150822	-0.193664	-0.142668	0.008333	-0.07
19/20	15	-0.149548	-0.174279	-0.163977	0.006658	-0.08
20/20	17	-0.156418	-0.174663	-0.155186	-0.025960	-0.09

10. Discussion

The experiment successfully highlighted several fundamental concepts in machine learning. Before training the numerical features were normalised for improving the model convergence of learning algo.

We can observe that single layer perceptron has limitations that it cannot solve non-linear problems like XOR logic as it can only draw a line and linear hyperplane. This banknote dataset is mostly linearly seperable and this made the learning algo smooth evaluations.

By comparing our implementation from scratch to the scikit learn's perceptron module, our implementation exposes the whole leaning process and makes it easier to understand how that works internally. while scikitlearn is more optimized for practical applications, better educational insights for the Perceptron learning algo can be done with the custom implementation.

11. Conclusion

The Single Layer Perceptron was successfully implemented from scratch and effectively classified the Banknote Authentication dataset with high accuracy. The experiment demonstrated the working of the Perceptron Learning Rule, including weight updates and convergence over multiple epochs. Overall, the implementation provided a clear understanding of binary classification using a perceptron and validated its effectiveness on linearly separable data.

12. References

1. F. Rosenblatt, “The Perceptron,” *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.